

Harry Potter And the Language Spell: Next Word Prediction and a comparison of Text Representation techniques.

Kartik Thakkar

Utah State University

Logan, Utah, USA

kartik.thakkar@usu.edu

Abstract

In Natural Language Processing, one of the rudimentary tasks is Language Generation, of which, Next Word Prediction is a problem type. In simple words, the problem can be described as *"Given a sequence of words, what will be the next word in the sequence?"*. The scholarship has a wide range of proposals that solve this in a variety of approaches: using different kinds of model architectures like Feed Forward Networks, Recurrent Neural Networks, Long Short Term Memory, or selecting from a variety of methods like Bag of Words, TF-IDF, Transformer Based Embeddings, etc for representing the input sequence in the most efficient way, and sometimes, combinations of both. This project aims to explore these techniques to learn and employ them on real datasets. The idea is to experience firsthand that the performance of a model built to solve this problem largely depends on the representation and quality of data it is trained on. In this project, we have attempted to predict the next word on data by following multiple approaches in building model architectures and representation of the text sequence these models are trained on. For performance evaluation, we used classification metrics like accuracy and cross-entropy loss for quantitative evaluation, and further, for the prediction of the best model based on these metrics, we used a manually created set of test sequences.

1 Introduction

1.1 Motivation

Predicting the next word in a sentence is a fundamental task in natural language processing (NLP) and is crucial for applications like text completion, dialogue systems, and language models. We have learned about RNNs and Text Representation in classes and wanted to explore their applications in the 'Next Word Prediction' type of problem. By using well-known scripts like the "Harry Potter"

movies, we aim to explore how the Next Word Prediction models can capture a well-known fantasy world's story details and themes. Understanding how language patterns are learned in such a context could help improve language models used in entertainment, writing assistance, and even character dialogue generation in the entertainment industry.

1.2 Position Overview

So far, we have majorly observed the next word prediction accuracies and losses of the model architectures on training and validation data sets. Although some slight improvements were seen in terms of these metrics from the models that we trained after our base model, their overall general results are still not what we are aiming for.

Currently, we are working on a new method (not mentioned in this paper since it is still in progress) to train a LSTM based model architecture by not using BERT embeddings as text representation technique. This is because we are not able to see any improvements even though we used LSTM based networks and they are known to perform better than the feed forward networks for language generation tasks.

This new method will most likely use a different method to represent text data before it is trained on any kind of model architecture.

1.3 Goal

The primary goal of this project is to develop a next-word prediction model that can relevantly predict the next word given a sentence that is linguistically similar to the language used in the Harry Potter Universe. Additionally, we want to explore different text representation techniques to analyze their effect on the quality of predictions. The model will learn from the vocabulary, structure, and context of the text that is processed from a dataset that was created using the movie scripts, which could lead to the next word for a sentence. We will compare

the performance of different embedding techniques to evaluate their effectiveness in predicting the next word.

1.4 Research Questions

- How does the style of the chosen model architecture relates to its ability of accurately predicting the next word?
- What is the effect of representation and preparation of input data on the accuracy and loss of the model in terms of prediction of the next word?

2 Related Work

Usage of Long Short Term Memory offers the advantage of understanding the context of the sentence in a better way in comparison to plain Neural Networks. In the works by (Ambulgekar et al., 2021), the next word prediction problem is solved using RNNs to help predict the 10 most probable next words. The work by (Irie et al., 2018), mentions use of LSTM hidden state representations of n-gram sequences for training an n-gram feed-forward network inspired us to choose our methods: Using LSTMs but training them on n-gram BERT embedding representations and using BERT embedding representations complete sequences to train feed-forward network. Moreover, hybrid representations have shown to be useful for other NLP tasks like recommendations as shown in (Polignano et al., 2021) where the authors used graph embeddings and contextualized word representations. This gave us the idea of using a combination of BERT and word2vec embedding representations in order to bypass the piece-wise tokenizations by BERT. Other works like that of (Al-Mubaid, 2003) directed us towards treating the problem of next-word prediction as a classification problem. This is done such that each of the unique words in the targets of the dataset forms a finite-sized corpus of words, many times, this corpus can be made up of thousands of words. Then, it specifically becomes a multi-class classification problem.

3 Methods

3.1 Choices and their Rationale

The methods used in this project are concerned with 3 important things: The model architecture being used, the technique being used to prepare the data that will be fed into the model architecture, and the

technique being used to represent the prepared data. **Why?** Because:

- The choice of model architecture affects how well the patterns in the text data are captured.
- The way the data is prepared affects the ability of the model architecture to find the patterns in the text data. Bad data preparation in terms of dependent and independent variables will hinder the model's ability to predict accurately because the data given to it is not in a specific format and hides the underlying patterns in the text.
- The text representation technique being used has a direct effect on the learning of the model since capturing the context of words improves the model's ability to learn when similar words appear in different contexts.

3.2 Technical Details

For model architecture, the variations we have worked on yet are models with LSTM layers and Bi-Directional LSTM layers. In these, for tuning the results, we tweaked the model architecture by adding and removing dropout and normalization layers to tackle overfitting on data.

For data preparation, we prepared our data (independent X and dependent y variables) majorly in two ways: first, considering n-grams of the text sequences where each $(n+1)^{th}$ word was the target. Second, for each input sequence, the last word was taken as the target.

For text representations, we have used a pre-trained transformer-based model BERT to generate embeddings for the prepared data. We are exploring other methods of representing the data for the future.

The primary evaluation metrics and hyperparameter values that we used to compare and train our models are described in Table 1.

Table 1: Metric and hyperparameter choices used for comparison and training of models

Metric	Choice
Optimizer	Adam
Learning Rate	0.001
Loss	Categorical Cross Entropy
Accuracy	Accuracy averaged over epochs

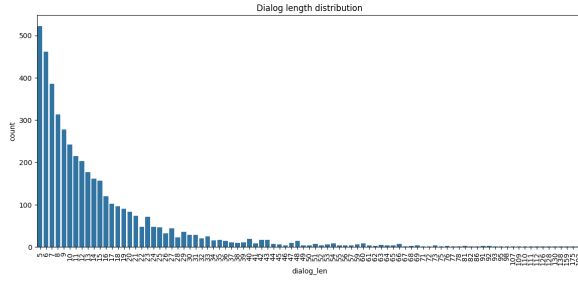


Figure 1: Dialog length distribution before shortening dialogs with large number of words

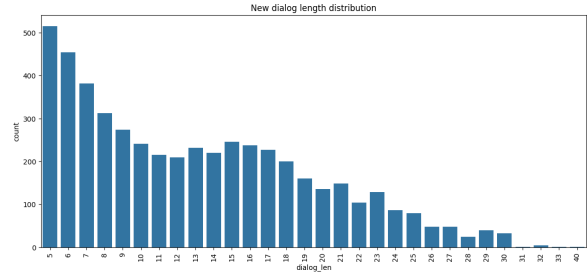


Figure 2: Dialog length distribution after shortening dialogs with large number of words

4 Experiments

- **Dataset** : The original raw dataset is a Kaggle dataset titled "Harry Potter Movies Dataset" that is primarily maintained by *Louis Chauvet*. The dataset contains 8 'csv' files for each part of the Harry Potter movie saga. Each file has the following columns : *Movie Name, Chapter Name (according to scripts), Character and Dialog*. For this project, we were only concerned with the *Dialog* column.
- **Primary Data Cleaning** :
 - First, all the contractions were expanded to their natural form so that each dialog is suitable for the embedding model to accurately represent it.
 - Second, all the non alphanumeric characters were removed and replaced with an empty space after which, complete text was converted to lower case.
 - After this preliminary cleaning, the dataset had a total of 4422 dialogs. Although the dialog length distribution was skewed towards <20 words per dialog, the data still had dialogs with as much as 192 words, which is difficult to work with. To deal with this, we created a custom method to split bigger dialogs into smaller sentences based on the number of words they had. After this, the dataset had almost 1000 new dialogs. The word length distribution before and after applying this method can be seen in the figures 1 and 2.
- **Expected Baseline Performance**: Since our goal is to compare the performance of different model architectures and also the effect of the way the data is fed into these model architectures, we are primarily going to use BERT

Embeddings to represent our inputs. To set a benchmark, we trained a regular Feed Forward Neural Network with 2 Linear Layers with Relu and Dropout Layers in between. The inputs X was complete sentence embeddings and y was the last word (label encoded for each sentence). We trained it for 100 epochs to set a baseline metrics that we wish to surpass with the subsequent experiments we try. The results were not great as the model overfitted on training data. However, for subsequent experiments, we aim to see improved metric values accuracy and cross entropy loss in comparison to the following metric values for our base model : 0.91, 0.74, 13.3, 0.14 for train loss, train accuracy, val loss and val accuracy respectively.

4.1 Methods Applied

We have trained some models so far with combinations of varying model architectures as well as the data they are trained on. These are described below:

1. **Base Model** : A feed-forward network architecture structured with the following layers: Linear $\rightarrow$ Relu $\rightarrow$ Dropout $\rightarrow$ Linear. Both Linear Layers had 128 hidden units and were trained for 100 epochs. The metric values are mentioned above.
2. **Improved Base Model**: To improve the performance of our base model, we decided to slightly increase its complexity by increasing the number of hidden units in the linear layers to 256. Better training performance was observed in comparison to base model but overall, it still overfitted. The train loss, train accuracy, val loss and val accuracy were 0.21, 0.94, 13.05 and 0.15 respectively. The model was trained on X and y

prepared through similar fashion as for base model.

3. **Bi-Directional LSTM Model + Hybrid Embeddings:** This model was structured as 2XLSTM layer $\rightarrow$ Linear Layer. The LSTM layers were bi directional and had 512 units each. The input for this model was prepared in a customized way described below:

- The BERT Tokenizer uses subword tokenization often resulting in tokens preceded by a # symbol. Because of this, a lot of tokens in the Tokenizer's vocabulary have the hash symbol. We believe that this might reduce the uniqueness of each token when it is represented in its embedding form.
- To fix this, for each token, we created a custom *token: embedding* map. We used the BERT Embedding for tokens that were in Tokenizer's vocabulary in pure form but for the Out-Of-Vocabulary (OOV) tokens, we used word2vec embeddings of same dimensions (768).
- Moreover, we created sequences based on n-grams so that the independent variable X contained n-gram subsequences of each dialog and y contained (n+1)th token for each subsequence.

Due to a large number of subsequences created, and a limited computational power and time, we only trained this model for 50 epochs. The train loss, train accuracy, val loss and val accuracy were 6.4, 0.02, 7.2 and 0.02 respectively. **Although LSTM-based models are known to perform better in sequence-based problems in comparison to simple Feed Forward Networks, these results were worse than the base model. Our speculation is that if trained for a longer duration, we might get better results as the computation involved in the learning process is very high due to the high dimensional nature of the data being passed to it.**

4. **Bi-Directional LSTM Model + Word-Index Representation:** Instead of using hybrid embeddings like in the previous model, we trained a tokenizer on the vocabulary of our data. Just like our previous model, we created sequences based on n-grams so that the

independent variable X contained n-gram subsequences of each dialog and y contained (n+1)th token for each subsequence.

The difference here was the representation of the data. Instead of high dimensional input data for the model that we believe requires superior computational resources to give the results expected from them, we used something far more simple yet effective - word: index representations of each word in the data.

Surprisingly, the metric values that we got were much better than all of the models we tried before. The model was trained for 100 epochs and the train loss, train accuracy, val loss and val accuracy were **0.80,0.78,10.46 and 0.11** respectively.

5. **Bi-Directional LSTM with a nn.Embedding Layer:** This model was structured as nn.Embedding layer $\rightarrow$ LSTM Layer $\rightarrow$ Dropout $\rightarrow$ LSTM Layer $\rightarrow$ Linear Layer. The LSTM layers were bi-directional. The first LSTM layer had 128 hidden units, and the second LSTM layer had 64 units. The Embedding Layer had an embedding dimension of 300.

The major difference with this model arrangement and definition in comparison to all the other variations we had tried before was as follows:

- Introduction of a nn.Embedding layer.
- Decreasing hidden units in the subsequent LSTM Layers. That is, the first LSTM layer had 128 hidden units and the second LSTM Layer had 64 units.

The rationale behind this choice was the average results from the previous model configurations, architectures, and data preparation strategies. The metric values that we got from this model were again, comparable to the 'top performing' models from before but it was opposite to the improvements we expected. The model was trained for 100 epochs, and the train loss, train accuracy, val loss and val accuracy were **0.80,0.78,10.46 and 0.11** respectively.

5 Revisions and addressing the previous issues

The *model 3* we trained had more than 40,000 subsequences each of which was padded to have 39 (the length of the longest sub-sequence) tokens each of which had an embedding dimension of 768 i.e. the data of shape before splitting into training and validation sets had a shape of (~40000,39,768) which significantly increased the computation time and was not feasible to work with limited computational power since we are using Google Colaboratory for this project.

This is why we revised **the form of the data the model is trained on while keeping the model architecture the same**. We used the whole data to create a vocabulary that includes all of the words in the data and created a word: index map to represent the subsequences. This was done with the help of a regular tokenizer offered by the *Keras* package. This not only solved the problem of piece-wise or subword tokenization that BERT Tokenizer produced, due to which we used hybrid embeddings but also allowed us to train the model for more epochs so that gradient descent optimized nicely.

The training and validation results were much better than any of the models we trained before. Although the validation metrics, were not as great as expected from a nicely trained model, they were better than other models (much less validation loss but similar validation accuracy)

However, even after transforming data into a word : embedding map for better representation, the training and validation accuracies seemed unsatisfactory. This is why, we added some variations in our next model (model 5). Model 5, performed better than the others and produced better training and validation accuracies (0.78 and 0.11) while both the losses remained comparable.

As mentioned in the last submission, we used this model to further test it on completely new data that we created manually. These test sentences were created with expectations from the best model to relevantly predict the next word based on the language used in these test sentences. This was done to qualitatively evaluate the performance of the model. It was expected that the best-performing model would capture the essence of the test sentence given to it. For example, if the test sentence has something that

gives away or insinuates that it is something that can be from the world of Harry Potter movies, the next word the model predicts must also follow along that context. The results are described in the 'Results' section.

5.1 Validation strategy

The latest model with the best training and validation metrics was further validated using completely new test sentences. These test input sentences were crafted manually and had some semantic connection with the language used in the input data. For a robust understanding, these test sequences were created such that they were of varying lengths. A few example test sequences with their expected next word are as follows:

- Sequence : *"drink the polyjuice"* Expected next word : *potion*
- Sequence : *"he who must not be"* Expected next word : *named*
- Sequence : *"wingardium"* Expected next word : *leviosa*
- Sequence: *"do you solemnly swear that you are up to no"* Expected next word: *good*
- Sequence: *"all you need is the golden"* Expected next word: *snitch*

The best model was tested on similar input sequences to evaluate the **quality** of the predicted next word. The best prediction would be the exact same word, a good prediction would be a similar word and the worst prediction would be something completely off.

6 Results

The quantitative results from training and validation are listed in Table 2.

Table 2: Improved Model's (model 5) Training and Validation Metrics

train_loss	0.79
train_acc	0.78
val_loss	10.31
val_acc	0.11

Table 3 displays the expected next word, the predicted next word, and the quality of the

prediction according to the prediction quality description in *Validation strategy* subsection above. Additionally, the table also has a column for the length of the input sequence in words.

Table 3: Expected Next Word, Predicted Next Word, Input Sequence Length and Prediction Quality

<i>reparo</i>	the	1	worst
<i>patronum</i>	patronum	1	best
<i>potion</i>	potion	3	best
<i>dumbledore</i>	mcgonnagall	5	good
<i>slytherin</i>	slytherin	2	best
<i>gryffindor</i>	gryffindor	3	best
<i>leviosa</i>	leviosa	1	best
<i>quidditch</i>	of	9	worst
<i>named</i>	going	5	good
<i>snitch</i>	snitch	5	best
<i>swear</i>	swear	2	best
<i>snape</i>	mr.	4	worst
<i>lived</i>	is	5	worst
<i>good</i>	problem	8	good

These metric values indicate that the fifth model is much better than other models (at least, the much higher training accuracy proves it). We speculate that although theoretically, the BERT and word2vec embeddings text representations are much more robust than simple word-index-based representation, there are 2 reasons when they might not be a good choice.

1. When using these models as pre-trained models for generating embeddings for niche data. In our case, the data we had contained a lot of words that are not common in the general English language (*for eg: Hogwarts, Gryffindor*), the representation might not be as good from these pre-trained models since these kinds of words are simply out of their vocabulary.
2. When the computational power is limited but the data is relatively large. When we created n-gram sequences for original dialogs, we ended up with thousands of subsequences which significantly increased the dimensionality of the data the model received thereby increasing the computational power. We used Google Colaboratory for this project and due to its random availability, it was really difficult to train the models that expected such a large dimensionality of data.

7 Discussion

Quality-wise, from the predictions of the next word on completely new sentences, this revised model performed well but not great. The following observations were made :

- For some test sentences, the predicted word was completely off from the expected word in terms of semantic similarity, a reason for this could be the test sentence itself was not very semantically similar to the language used in the movies.
- For smaller test sentences, the model did accurately produce a relevant next word indicating the ability to learn from the context of the data.
- For bigger sentences, the predicted next word was completely off. This was expected since this '*context-problem*' is what led to the development of advanced methods like *encoder-decoder architectures and transformers*.

Overall, the results from this latest method are better in comparison to the other methods we tried before. However, they are not good in terms of what is expected of a model like this.

8 Limitations and Future Work

The major limitation we had was the handling of large dimensional data with limited computational power. Another minor personal limitation for us was no prior experience with handling such large data with Pytorch.

Another area of improvement that we plan on working on is the qualitative accuracy of the next word prediction. Although there are methods like stop-word removal to deal with common words, we still believe that these words add to the structure of the language and should not be removed. A few predictions were just words like *the*, *of*, etc. In the future, we plan on working with better sequence-to-sequence methods like encoder-decoder mechanisms or transformers that are proven to be best for tasks like these.

References

- Hisham Al-Mubaid. 2003. Context-based word prediction and classification. In *Proceedings of the 18th International Conference on Computers and their Applications CATA*, volume 2003, pages 384–388. Citeseer.
- Sourabh Ambulgekar, Sanket Malewadikar, Raju Garande, and Bharti Joshi. 2021. Next words prediction using recurrent neural networks. In *ITM Web of Conferences*, volume 40, page 03034. EDP Sciences.
- Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.
- Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.
- Alex Graves. 2015. Generating sequences with recurrent neural networks. *Computing Research Repository*, arXiv:1503.06733. Version 2.
- Kazuki Irie, Zhihong Lei, Ralf Schlüter, and Hermann Ney. 2018. Prediction of lstm-rnn full context states as a subtask for n-gram feedforward language models. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6104–6108.
- Bowen Liu, Bharath Ramsundar, Prasad Kawthekar, Jade Shi, Joseph Gomes, Quang Nguyen, Stephen Ho, Jack Sloane, Paul Wender, and Vijay Pande. 2017. Retrosynthetic reaction prediction using neural sequence-to-sequence models. *ACS Central Science*, 3.
- Al Amin Md Robiul Islam and Aniqua Nusrat Zereen. 2024. Enhancing bangla language next word prediction and sentence completion through extended rnn with bi-lstm model on n-gram language. *IEEE*.
- Marco Polignano, Cataldo Musto, Marco de Gemmis, Pasquale Lops, and Giovanni Semeraro. 2021. Together is better: Hybrid recommendations combining graph embeddings and contextualized word representations. In *Proceedings of the 15th ACM Conference on Recommender Systems, RecSys '21*, page 187–198, New York, NY, USA. Association for Computing Machinery.
- N. Aggarwal P. Kaur R. Sharma, N. Goel and C. Prakash. 2019. Next word prediction in hindi using deep learning techniques. *International Conference on Data Science and Engineering (ICDSE)*.
- Peter Haider Steffen Bickel and Tobias Scheffer. 2005. Predicting sentences using n-gram language models. *HLT/EMNLP*.